

Complexity part 4. Abstractions.

In software development, abstraction is one of our most powerful tools — it allows us to hide irrelevant details and work with simplified mental models. It's how we manage complexity: instead of thinking in terms of bits and bytes, we work with concepts like buttons, lists, users, or invoices. The goal of abstraction is to make systems easier to understand, use, and evolve by reducing the amount of information we need to hold in our heads at once.

But abstraction is also a double-edged sword. When done poorly — too early, too generically, or in the wrong context — it can increase complexity instead of reducing it. Layers of indirection, leaky contracts, mismatched models, and unnecessary generalization all make the system harder to reason about. This article explores how abstraction helps and harms, how to recognize bad abstractions, and how to build the right ones for the problems at hand.

Modeling Reality

Abstractions are all around us — not just in software, but in how we perceive and understand the world. In the physical world, we work with layers of abstraction to make sense of complex systems. At the smallest scales, we study atoms, molecules, and cells. At higher levels, we talk about tissues, organs, organisms, and ecosystems. In physics, we might go from quantum particles, to materials, to human-scale mechanics, all the way up to planetary and galactic systems. We use different models for each of these layers because it would be overwhelming — and unnecessary — to consider all layers at once. We don't need particle physics to treat a broken leg.

Abstractions are always adjusted to the **context**. Consider a country on a map: the same country can be represented in vastly different ways depending on what you're trying to understand. A political map highlights borders and capitals. A population density map reveals demographic patterns. A topographic map shows terrain. All of these are abstractions of the same reality — but each is tuned for a specific purpose.

Software development follows the same pattern. When we build systems, we are modeling reality. Everything we create — data types, services, app modules, even full architectures — is a model of something. A `User` data type models a real person (or, more precisely, the aspects of that person that our system cares about). A `NotificationService` models the process of informing users. An entire app might model a shopping experience, a logistics pipeline, or a healthcare workflow.

These models, like all abstractions, intentionally leave out some details to make the system manageable. What we choose to represent — and what we choose to ignore — is guided by context.

The art of abstraction lies in making two essential decisions: first, choosing the right context — understanding what question we’re trying to answer and how we intend to work with the abstraction; and second, deciding which details are important enough to include and which can be safely left out, based on that chosen context.

When we get those two things right, abstraction reduces complexity. When we don’t, it often does the opposite.

Common Abstraction Pitfalls

Even though abstraction is a tool for managing complexity, it can easily become a source of complexity itself when misapplied. Here are some of the most common ways abstractions go wrong.

1. Wrong Level of Abstraction

An abstraction that operates at the wrong level can either oversimplify or overcomplicate the problem. When it’s too shallow, it omits important details and fails to support real-world usage. When it’s too deep, it tries to cover every possible scenario — cluttering the interface with edge-case handling, parameters, and configuration options that most consumers don’t need.

This often stems from a desire to generalize too early — seeing two similar bits of logic and merging them into a shared abstraction. But surface-level similarity doesn’t always mean shared purpose. The result is a brittle abstraction that satisfies no use case fully, and becomes harder to evolve as requirements change.

Signs this is happening: The abstraction claims to be general-purpose but only supports a subset of use cases in practice. It contains branching logic or edge-case handling that grows over time. It’s difficult to use correctly without reading the implementation. It feels like a compromise: not expressive enough for one case, too verbose for another. Every time a specific use case changes, you’re forced to touch the abstraction. You find yourself thinking: “This would be easier if each case had its own implementation.” You’re writing more code to support the abstraction than to solve the original problem. Refactoring it feels risky because it affects many unrelated parts of the codebase.

Examples: A `Shape` protocol that assumes all shapes have corners — then you need to add a `Circle`, and everything breaks. A `BaseViewModel` with dozens of hooks to accommodate multiple screens, where each screen uses only a few of them — and often in incompatible ways. A `Config` object that tries to hold all environment settings, screen parameters, and user preferences in one place — but becomes unreadable and impossible to safely evolve. A `FormField` abstraction designed to cover every kind of input (text, number, date, dropdown, toggle), but the API becomes so bloated with configuration flags that each new field type needs multiple `if`-statements to function properly.

*Hint: Before abstracting, ask: **Are these really the same thing? Or are they just accidentally similar right now?***

2. Redundant Abstraction

There are many cases where an abstraction is created without a real need for one. This often happens in the form of **premature abstraction** — introducing layers before real duplication, variation, or friction appears. The intent may be to “do it right from the start” or to make future changes easier. But in reality, these abstractions are usually based on guesses — guesses about what might be reused, or what could change. The problem is, the future rarely plays out exactly as we expect.

Signs this is happening: The abstraction is only used once — or has no meaningful reuse. It’s harder to onboard someone into the abstraction than to understand the actual logic. You use passthrough functions, parameters, or entire layers that don’t add value. You often bypass the abstraction or re-implement similar functionality elsewhere. You can’t explain why it exists without saying “just in case” or “it might be reused.” Understanding the code requires jumping between multiple files and types. The abstraction has more structure than substance — its logic is trivial or nonexistent.

Examples: A `Logger` protocol introduced to allow log swapping — but no one ever uses any implementation other than `ConsoleLogger`. A `StorageManager` class that wraps `UserDefaults` with identical method signatures, adding no behavior beyond delegation. A `ViewModelProtocol` created for testability, but only one concrete implementation exists — and the test uses the real one anyway. A `NetworkClient` interface introduced early in the project, even though only one backend is used and no alternative is planned.

Hint: Prefer concrete clarity over abstract purity.

3. Leaky Abstractions

A leaky abstraction is one that fails to fully hide the complexity it was meant to encapsulate. From the outside, the abstraction should offer a clean, predictable interface — shielding its users from internal details. But with a leaky abstraction, you find yourself digging into the implementation just to understand how to use it correctly or safely.

Joel Spolsky’s *Law of Leaky Abstractions* puts it clearly: “*All non-trivial abstractions, to some degree, are leaky.*” While some degree of leakage may be inevitable, it becomes a real problem when the abstraction **pretends** to simplify something but still forces you to understand what’s underneath. Instead of reducing cognitive load, it increases it — with a false sense of simplicity.

Signs this is happening: You need to understand internal behavior or edge cases to use the abstraction correctly. The abstraction exposes implementation-specific concepts in its API. You

frequently open the source to “see what it’s doing under the hood.” You find yourself passing raw internals (like file paths, query strings, or framework-specific data) through an otherwise abstracted interface. You must remember undocumented caveats or usage patterns to avoid breaking things. You get unexpected bugs from valid usage — because *how* it works matters more than *what* it claims to do. The abstraction doesn’t match the mental model it tries to present.

Principles for Better Abstractions

Abstractions are not inherently good or bad — their quality depends on how well they fit the problem they’re trying to solve. When used appropriately, they reduce cognitive load, isolate complexity, and improve code reuse. When misused, they obscure logic, create fragility, and slow teams down. The key is to approach abstraction deliberately and iteratively — not dogmatically.

- **Let abstractions emerge.** Don’t invent abstractions too early. Let them arise naturally from repeated patterns or real-world friction.
- **The name matters.** A precise name forces clarity. If you struggle to name your abstraction, you haven’t figured out what it really does.
- **Design for now, not for maybe.** Build the abstraction that solves today’s problem well. Reuse should be a byproduct of clarity.
- **Hide what doesn’t need to be known.** Keep the interface minimal. Expose only what’s essential.
- **Utilize the separation of concerns principle.** If your abstraction is doing too much, split it into focused, single-purpose units.
- **Design by subtraction.** Ask yourself: “What can I remove and still have this work?” Good abstractions get simpler over time.
- **Avoid passthrough layers.** If an abstraction only delegates without adding behavior, remove it.
- **Reassess regularly.** Abstractions that were once useful can become barriers. Revisit them when requirements change.
- **Don’t fear repetition.** A little duplication is better than the wrong abstraction. Try inlining first.
- **If it leaks — rethink it.** A leaky abstraction is often a sign of wrong boundaries or misunderstood complexity.
- **Document the purpose.** Every abstraction should have a reason to exist. If you can’t explain it in one clear sentence, reconsider.

Conclusion

Abstractions are one of the most powerful ways we manage complexity in software projects. By hiding unnecessary details and organizing logic around meaningful concepts, they help us reason about systems more effectively and reduce the cognitive load required to work within them. **Good**

abstractions do more than hide details — they shape how we think, debug, and build. They influence the mental models we form, the questions we ask, and the mistakes we avoid.

Like architecture, abstraction is not something we get right once — it's something we adapt over time. The best abstractions are responsive, practical, and grounded in the real needs of the system. They reduce local complexity without introducing global confusion. When done well, abstraction becomes not just a technical device, but a strategic tool for building software that's easier to understand, evolve, and trust.